

## Professor Code Review Pack

# LawCompass 최종 구조 검토 및 발표 자료

다섯 가지 구조 문서가 요구하는 경량 MSA, Agent/MCP-like, Task-Plan-Goal, 표준 MCP 보류 결정, 프로젝트 원칙 준수 기준을 현재 저장소가 만족하는지 점검한 코드리뷰용 자료다.

**작성 기준**

2026-06-18

**브랜치**

main / origin 동기화

**검토 범위**

문서, Agent, MCP-like, 배포

**결론**

핵심 구조 만족

## 1. 최종 판정

현재 LawCompass는 다섯 구조 문서의 핵심 방향을 충족한다. 단, 완전한 표준 MCP 서버/클라이언트나 독립 Multi-Agent 프로세스가 아니라, Docker Compose 기반 경량 MSA형 모노레포 위에 Agent 내부 role-based specialist pipeline, MCP-like tool registry/executor, Task-Plan-Goal packet/trace 구조를 결합한 형태로 설명해야 한다.

구조 기준 만족

문서-코드 싱크 통과

Agent 계약 테스트 통과

SRP watchlist 경고 존재

표준 MCP 도입 보류

**교수님께 말할 핵심:** 표준 MCP를 붙였다고 과장하지 않고, 현재 문제에 맞게 내부 MCP-like executor와 Agent packet 구조를 먼저 안정화했다. 외부 tool server나 다중 host 재사용 요구가 생기면 표준 MCP로 확장할 수 있게 pilot 계약을 남겼다.

## 2. 다섯 구조 문서 만족 여부

| 문서                                      | 핵심 요구                                                                             | 현재 만족 근거                                                                                                                        | 판정          |
|-----------------------------------------|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|-------------|
| AGENT_MCP_TASK_PLAN_GOAL_ROADMAP.md     | role-based specialist pipeline, MCP-like registry/executor, Task-Plan-Goal packet | agent_contracts.py ,<br>planner.py ,<br>agent_goal_aggregator.py ,<br>tool_registry.py ,<br>tool_executor.py 존재. 관련 테스트 73개 통과. | 만족          |
| PROJECT_PRINCIPLE_COMPLIANCE_BACKLOG.md | 문서 흐름, 보안, 책임 경계, SRP watchlist, fallback/finality 관리                             | check_principle_compliance.py 통과. SRP watchlist는 경고로 남아 있으나 "수정 시 분리" 전략과 일치.                                                   | 만족<br>경고 보존 |
| STANDARD_MCP_DECISION.md                | 표준 MCP 즉시 도입 보류, 내부 executor 유지, 도입 trigger 명시                                    | standard_mcp_gate.py 와 테스트가 trigger 조건 없을 때 내부 계층 유지 결정을 검증.                                                                    | 만족          |
| STANDARD_MCP_PILOT_DESIGN.md            | search_knia_json_rag_tool read-only pilot, production runtime 미변경                 | standard_mcp_pilot.py 가 adapter contract를 생성하고 runtime 미변경을 검증.                                                                 | 만족          |
| FUTURE_MSA_MCP_AGENT_EVOLUTION.md       | 후속 전환 방향은 남기되 현재 안정화 범위를 깨지 않음                                                    | 컨테이너 분리 구조 유지. 독립 Agent process, Kubernetes, 서비스별 DB 분리는 후속 검토로 보류.                                                             | 만족          |

카카오톡 받은 파일 원본과 저장소 문서는 해시가 다르다. 저장소 문서는 현재 docs/ 구조와 링크 기준에 맞게 정리된 반영본이므로 프로젝트 현재 구조에 맞게 최신화된 상태로 보는 것이 맞다.

### 3. 현재 최종 아키텍처

사용자  
자연어/영상

Vue 3  
Frontend

Fastify  
Gateway

Redis Streams  
Queue

Python  
Worker

FastAPI  
Agent

PostgreSQL  
pgvector

#### Frontend

사용자 입력, 영상 업로드, 대분류/질문 UI, 결과 카드, KNIA/법률 근거 표시.

#### Gateway

인증, 세션, 공개 API, DB 저장, Redis queue 생성, Agent 내부 호출, report 조립.

#### Worker

영상 전처리, ffmpeg/ffprobe 대표 프레임 추출, OpenAI/YOLO 보조 관찰값 생성.

#### Agent

입력 정규화, 직접 충돌 대상 판정, KNIA/RAG 검색, 과실비율/가감요소 분석.

## 4. Agent 내부 동작 방식

1. 자연어, structured facts, 영상 metadata를 합쳐 분석 가능한 fact로 정리한다.
2. 직접 충돌 대상을 먼저 판정해 KNIA 대분류를 고정한다.
3. 영상 관찰값은 후보/확인/확정/충돌/무시 상태로 분리한다.
4. 대분류가 정해지면 다른 대분류 chart/RAG가 최종 결과에 섞이지 않게 party guard를 적용한다.
5. KNIA structured chart를 우선 검색하고 같은 chart 또는 같은 대분류 RAG만 사용한다.
6. 과실비율은 base fault, final fault, range, applied/unknown adjustment, conditional outcome으로 분리한다.
7. 사용자 화면에는 raw trace와 내부 key를 숨기고 참고/조건부/추가 확인 필요 표현으로 변환한다.

핵심은 AI가 과실 숫자를 임의 생성하지 않는다는 점이다. 숫자는 구조화 KNIA chart와 deterministic adjustment policy를 우선하고, 근거가 부족하면 reference-only 또는 needs-review로 낮춘다.

## 5. MCP-like 구조 설명

| 항목          | 현재 구현                                                                                             | 교수님 질문 대응                                         |
|-------------|---------------------------------------------------------------------------------------------------|---------------------------------------------------|
| Tool schema | <code>MCPToolSpec</code> 로 name, description, input/output schema, scope, timeout, side effect 정의 | 표준 MCP 전환 전에도 tool 계약과 권한을 명시적으로 관리한다.            |
| Executor    | 입력 검증, scope 검증, timeout, output schema 검증, failure packet 생성                                     | raw exception을 사용자 결과에 섞지 않고 안전한 실패 packet으로 남긴다. |
| Gate        | 외부 tool 수, 표준 client 요구, 보안 격리 요구가 생길 때만 도입 trigger                                               | 현재 규모에서 표준 MCP runtime은 운영 복잡도 대비 이득이 작아 보류했다.    |
| Pilot       | <code>search_knia_json_rag_tool</code> 을 future adapter compatibility 대상으로 설계                     | read-only tool부터 전환 가능성을 검토하는 안전한 migration 전략이다. |

**발표 금지 표현:** 표준 MCP 구현 완료, 외부 MCP 서버 연동 완료, 독립 Multi-Agent 프로세스 완성.

## 6. 코드리뷰 Findings

### Critical/High 결함

명확한 결함 발견하지 못함

문서 링크, 문서-코드 싱크, Agent/MCP 계약 테스트 기준으로 다섯 구조 문서의 핵심 요구와 실제 코드가 어긋나는 치명적 결함은 발견하지 못했다.

### Medium: SRP watchlist 경고

| 파일                                                | 라인 수 | 후속 분리 방향                                                              |
|---------------------------------------------------|------|-----------------------------------------------------------------------|
| apps/gateway/src/lib/report-composer.ts           | 2894 | summary, fault, evidence, KNIA, presentation mapper 분리                |
| apps/frontend/src/composables/useCaseWorkspace.ts | 1039 | state, upload, polling, hydration composable 분리                       |
| apps/agent/app/services/knia/knia_matcher.py      | 1181 | candidate retrieval, party/scenario guard, scorer, fallback policy 분리 |
| apps/worker/worker/frame_analysis.py              | 1439 | provider adapter, prompt builder, retry policy, usage tracker 분리      |
| apps/worker/worker/yolo_frame_analysis.py         | 1103 | detector runner, event selector, observation mapper 분리                |

## 7. 검증 결과

| 검증               | 명령                                                           | 결과                                                 |
|------------------|--------------------------------------------------------------|----------------------------------------------------|
| 문서-코드 싱크         | <code>python scripts/check_document_code_sync.py</code>      | <b>passed</b> required paths 56, snippet groups 19 |
| Markdown 링크      | <code>python scripts/check_markdown_links.py --strict</code> | <b>passed</b> checked links 49, broken 0           |
| 원칙 준수            | <code>python scripts/check_principle_compliance.py</code>    | <b>passed</b> check count 4, failed 0              |
| Agent/MCP 계약 테스트 | <code>cd apps/agent; python -m pytest ... -q</code>          | <b>73 passed</b>                                   |
| SRP watchlist    | <code>python scripts/check_srp_file_sizes.py</code>          | <b>warning</b> 9 files over threshold, non-failing |

루트에서 Agent pytest를 실행하면 `ModuleNotFoundError: No module named 'app'`가 발생한다. 이는 실행 위치/PYTHONPATH 문제이며, `apps/agent`에서 실행한 실제 검증은 통과했다.

## 8. 발표용 설명 흐름

- 문제:** 교통사고 사용자는 사고 사실, 영상, 보험 기준, KNIA 기준을 따로 확인해야 한다.
- 구조:** 화면, API, 영상 처리, AI 판단, DB/RAG를 컨테이너 단위로 분리한 경량 MSA형 모노레포다.
- 핵심 설계:** 사고 대분류와 직접 충돌 대상을 먼저 고정해 잘못된 KNIA 기준 혼선을 막았다.
- RAG:** 구조화 KNIA chart를 우선 선택하고, 선택된 chart 또는 같은 대분류의 RAG chunk만 검색한다.
- Agent:** 완전 독립 Agent process가 아니라 역할 기반 specialist result와 consensus packet을 붙인 내부 pipeline이다.
- MCP:** 표준 MCP runtime은 보류하고, 내부 MCP-like registry/executor로 schema, scope, timeout, failure packet을 관리한다.
- 품질:** finality, 조건부 결과, 표시 sanitizer, regression test로 근거 없는 확정 판단을 제한한다.
- 한계:** 일부 중심 파일이 크며, 표준 MCP/독립 Agent process/Kubernetes/서비스별 DB 분리는 후속 과제다.

## 9. 교수님 예상 질문과 답변

### Q. 왜 완전한 MSA가 아니라 경량 MSA형 모노레포인가?

현재 규모에서는 서비스별 독립 저장소와 DB 분리보다 컨테이너 단위 책임 분리와 명확한 API/DTO 계약이 더 실용적이다. Frontend/Gateway/Agent/Worker/PostgreSQL/Redis는 분리했지만 운영 복잡도를 줄이기 위해 Docker Compose 기반 단일 배포 단위로 유지했다.

### Q. MCP를 구현한 것인가?

표준 MCP 서버/클라이언트를 구현한 것은 아니다. 내부 MCP-like tool registry/executor를 구현했고, 표준 MCP 도입 조건과 read-only pilot adapter 설계를 문서화했다. 현재는 내부 executor가 production source of truth다.

### Q. AI가 과실비율을 마음대로 생성하지 않는가?

과실 숫자는 구조화 KNIA chart와 deterministic adjustment policy를 우선한다. LLM은 근거 설명과 요약을 보조하고, base fault가 없거나 review\_required이면 reference-only 또는 needs-review로 낮춘다.

### Q. 영상 분석이 사고 결론을 바로 내리는가?

아니다. Worker는 프레임과 객체/장면 관찰 후보를 만들고, Agent는 이를 사용자 입력, KNIA, 법령 근거와 함께 조율한다. 보행자나 자전거가 화면에 보인다는 이유만으로 사고 당사자로 승격하지 않는다.

### Q. 현재 가장 큰 리스크는?

일부 핵심 조립 파일의 규모, KNIA/판례 원문 coverage, GitHub Pages가 정적 프론트만 제공한다는 배포 한계다. 이들은 모두 문서와 후속 백로그에 분리해 두었다.

## 10. 최종 요약

### 구조

경량 MSA형 모노레포 + Agent 내부 specialist pipeline + MCP-like executor.

### 근거

KNIA structured chart와 RAG를 대분류/시나리오 기준으로 제한 검색.

### 품질

Task-Plan-Goal, finality, 조건부 판단, trace/usage/failure 관측성, 회귀 테스트.

### 결론

다섯 문서의 구조 기준을 만족한다. 구현한 것과 후속 전환 후보를 명확히 분리해 설명한다.